Introduction to Robotics: Localization Part 2 – Mapping

Shuai Zhang

Department of Computer Science
University of Central Oklahoma

Credit to the book Introduction to Autonomous Mobile Robots, by Roland Siegwart and Illah Nourbakhsh

Introduction 1

- In the previous part, we know there are two coordinate systems
  - One from the robot's view: local coordinate system
    - The location information maintained by the robot
  - One from the global observer's view: global coordinate system
- There are cumulative errors in the local view
- Initially, usually we need to calibrate the coordinate (see ackerbot.java example)
- After while, we also need to update the location information to reduce the errors
  - How? A landmark? A map?

Introduction 2

- With the map, the compass, Christopher Columbus still got “lost” when he tried to find a new route to India, China and Japan.
  - The shorelines / islands could be used as the landmarks
  - Unexpected long travelling distance meant something was wrong: they needed to correct the error
  - Based on the map, they could have a try
  - Instead they found the new continent
- The map is not accurate?
  - No one had sailed there before: the map needs to be updated

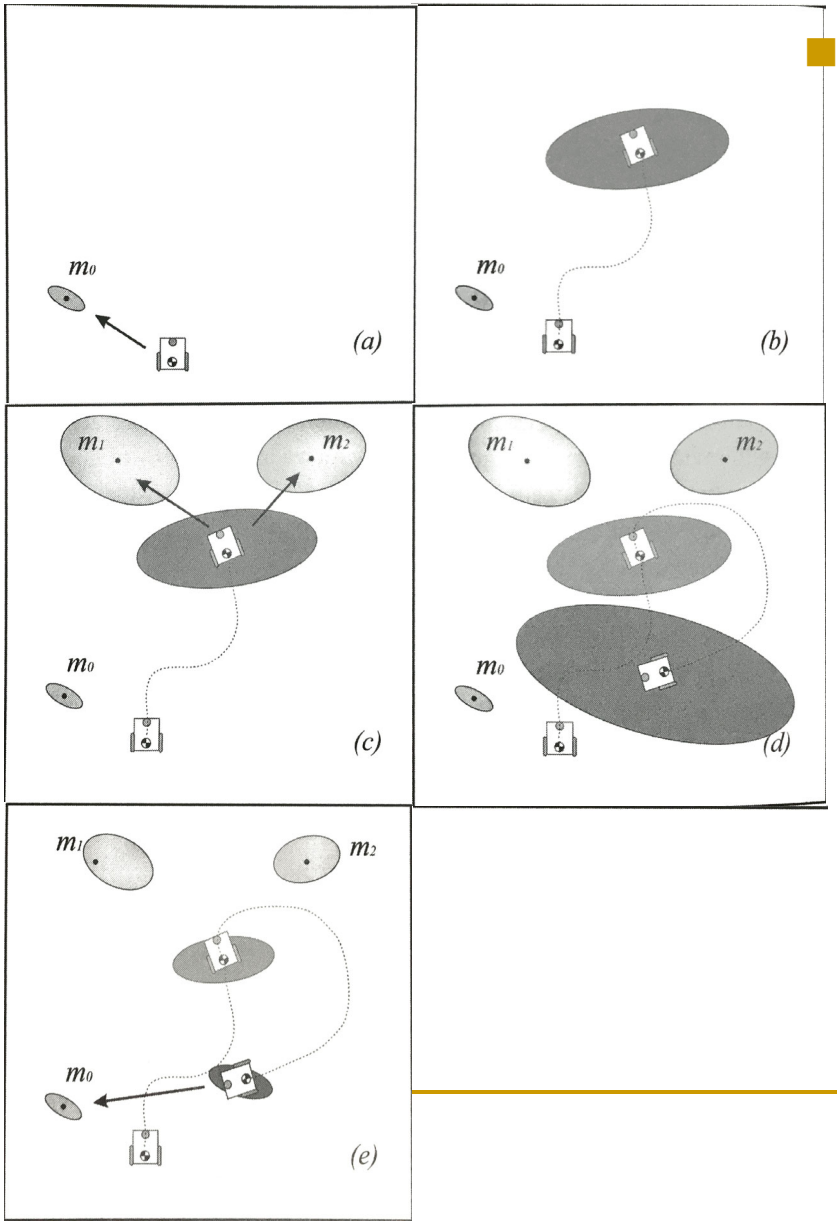
Introduction 3

- Get the location by the map first? Or make the map based on the current location first?
  - This is similar as the chicken first or the egg first
- How about we work on them simultaneously?
  - Simultaneous Localization and Mapping (SLAM)
  - Landmarks can be used to calibrate the location and the map
    - Visited landmarks' locations can be updated
    - Visited landmarks can also help reduce the uncertainty of the current location
    - New landmarks' locations can be added to the map

SLAM 1

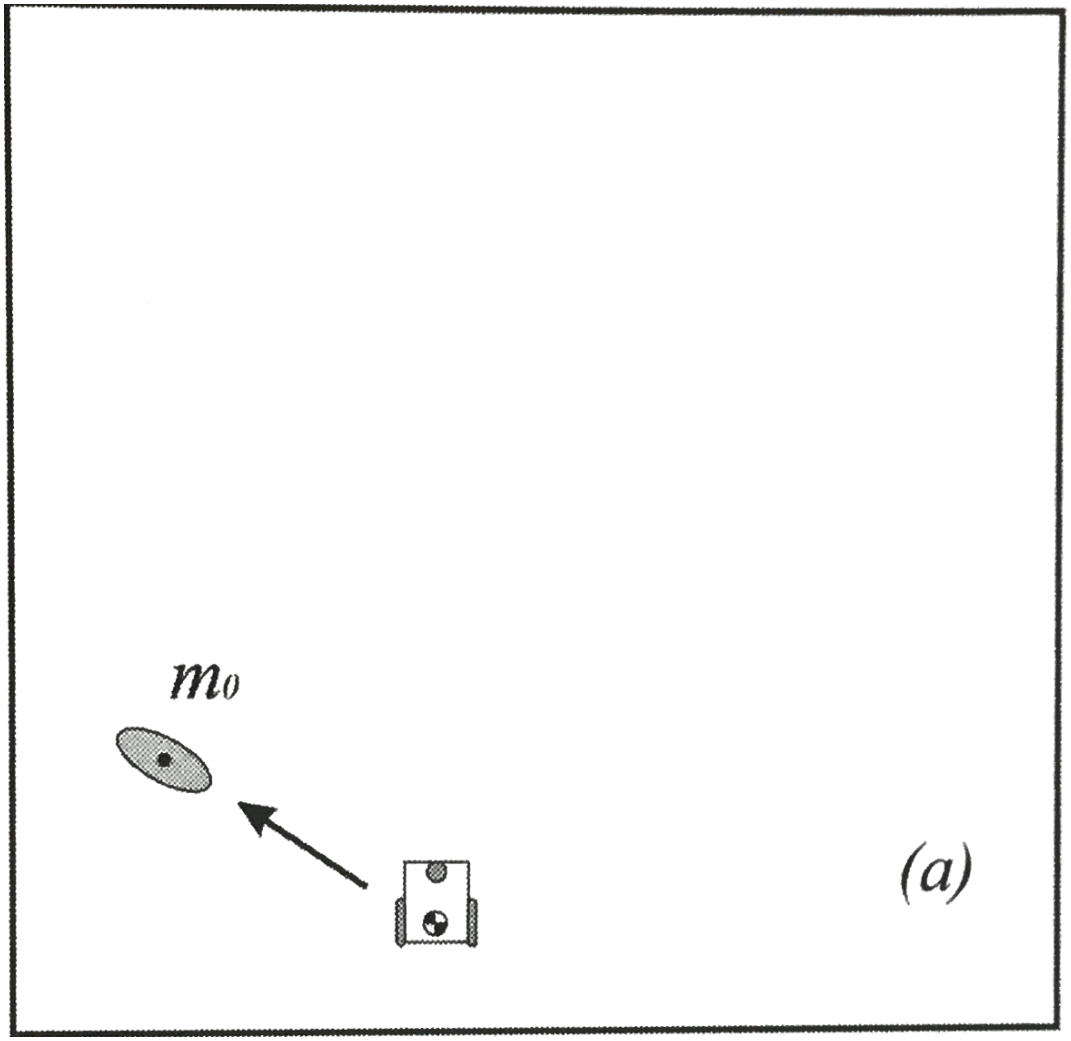
- SLAM is an infrastructure, not a standard algorithm: different features and algorithms can be applied in this infrastructure
  - E.g. graph-based, Particle-Filter-based SLAM, etc.
- Openslam contains the most comprehensive lists of SLAM software: https://openslam-org.github.io/
  - Most open source implementations are in Matlab and C/C++: we can focus on C/C++ implementations
  - One Particle-Filter-based SLAM from Duke University: https://users.cs.duke.edu/~parr/dpslam/

SLAM 2



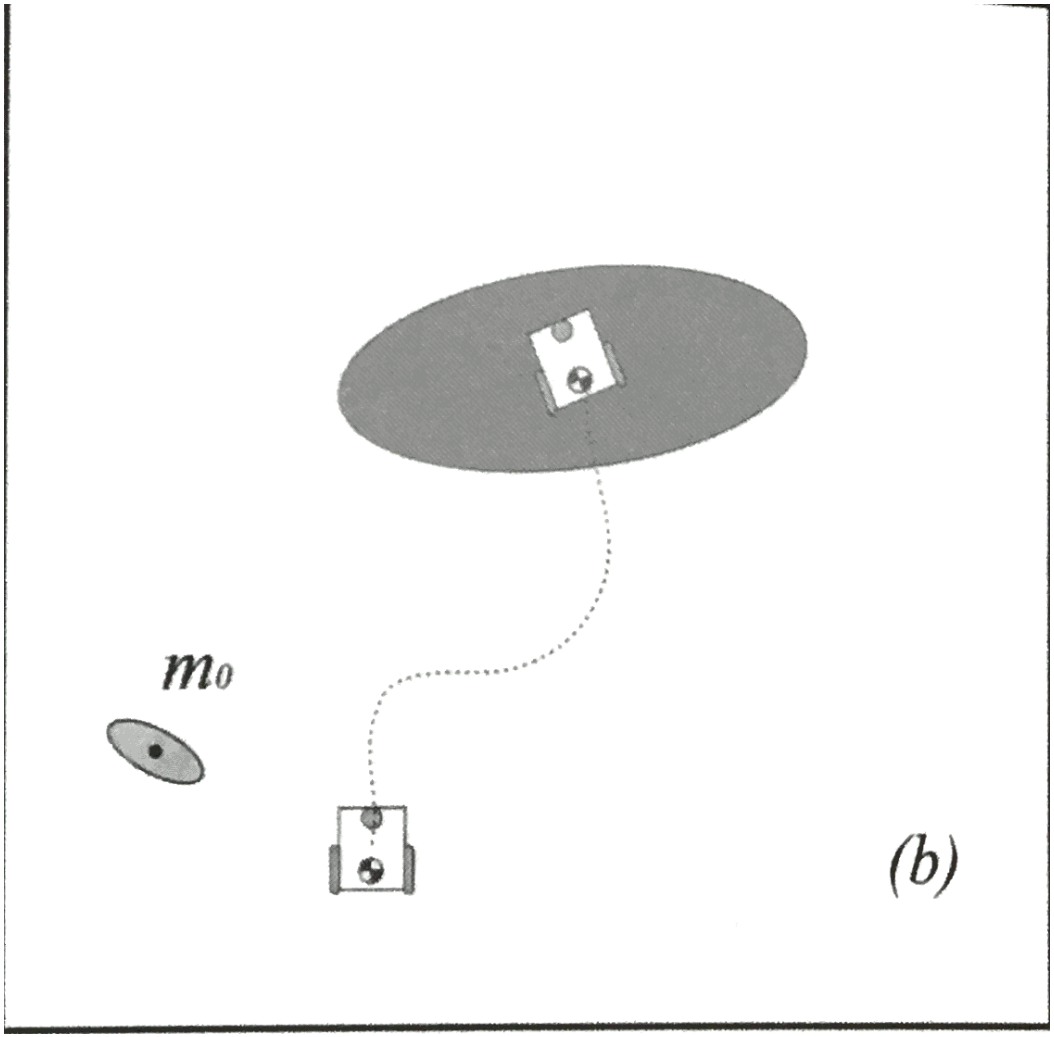
- Illustration of SLAM
  - There are propagated uncertainties through time
    - Big ellipse means large uncertainty
    - Without calibration or correction from the landmarks, the errors will grow
    - In a gaussian distribution, the standard deviation will be bigger through time
    - A revisit to previous landmark can reduce the uncertainty

SLAM 3



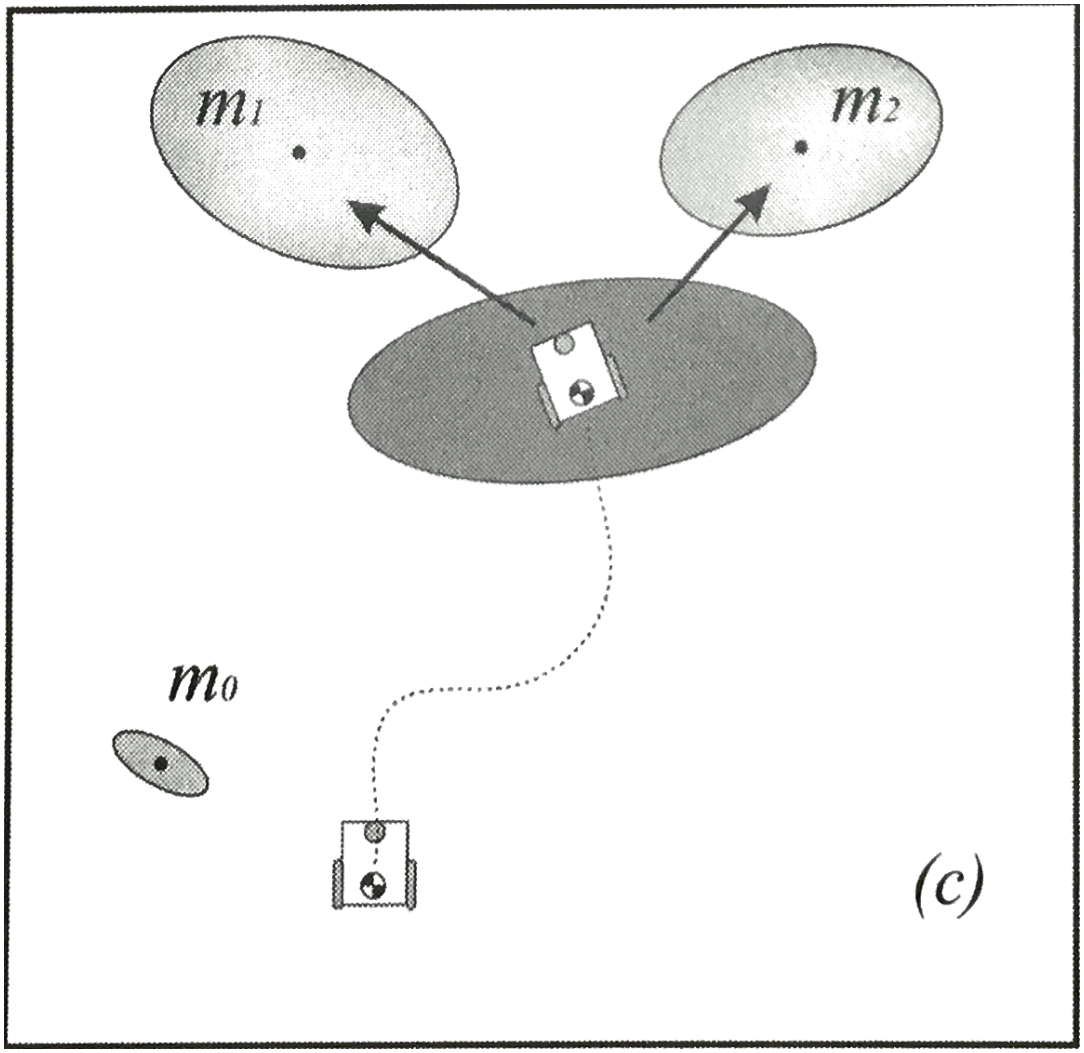
- Step (a)
  - The robot observes a landmark m_0, which is mapped with an uncertainty, based on the exteroceptive sensors.

SLAM 4



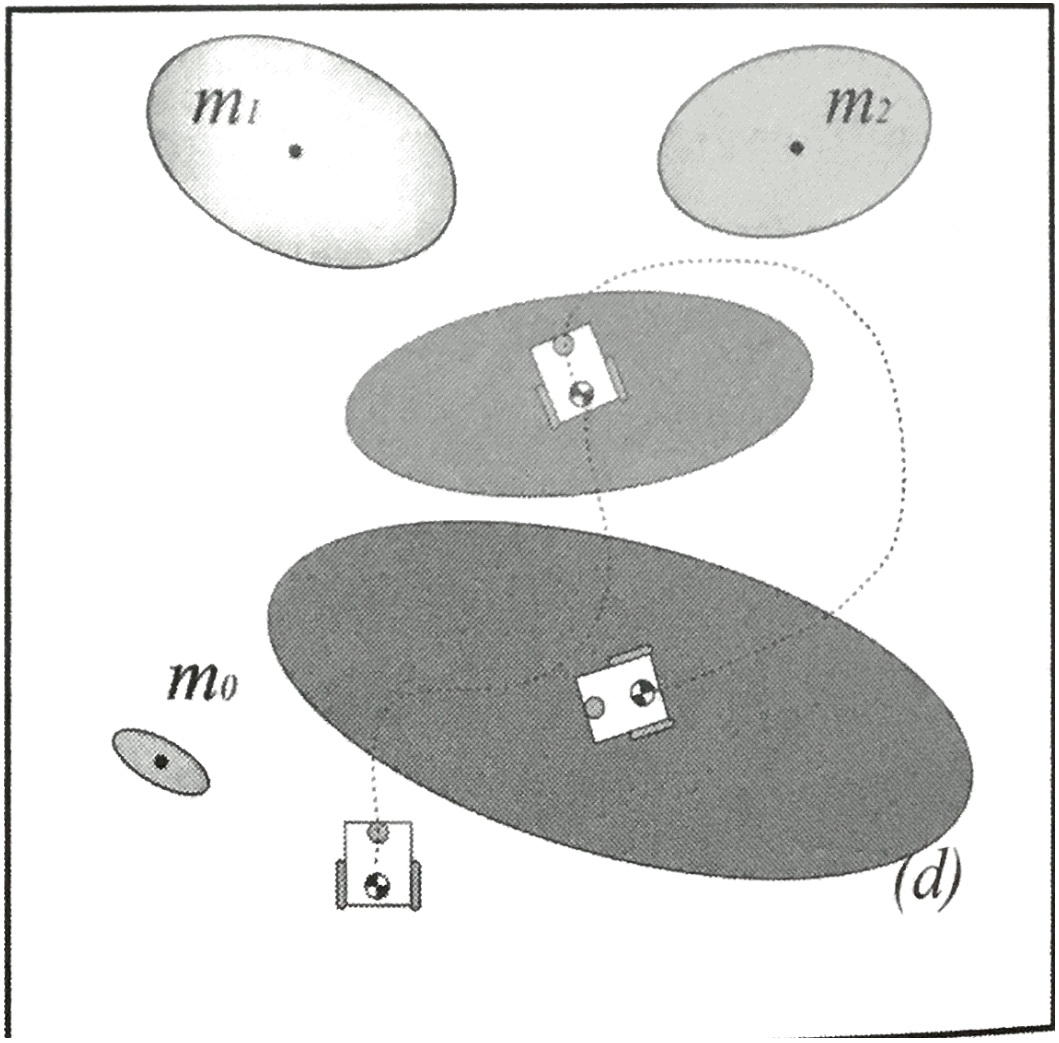
- Step (b)
  - As the robot moves, the pose uncertainty increases, under the effort of the errors introduced by the odometry.

SLAM 5



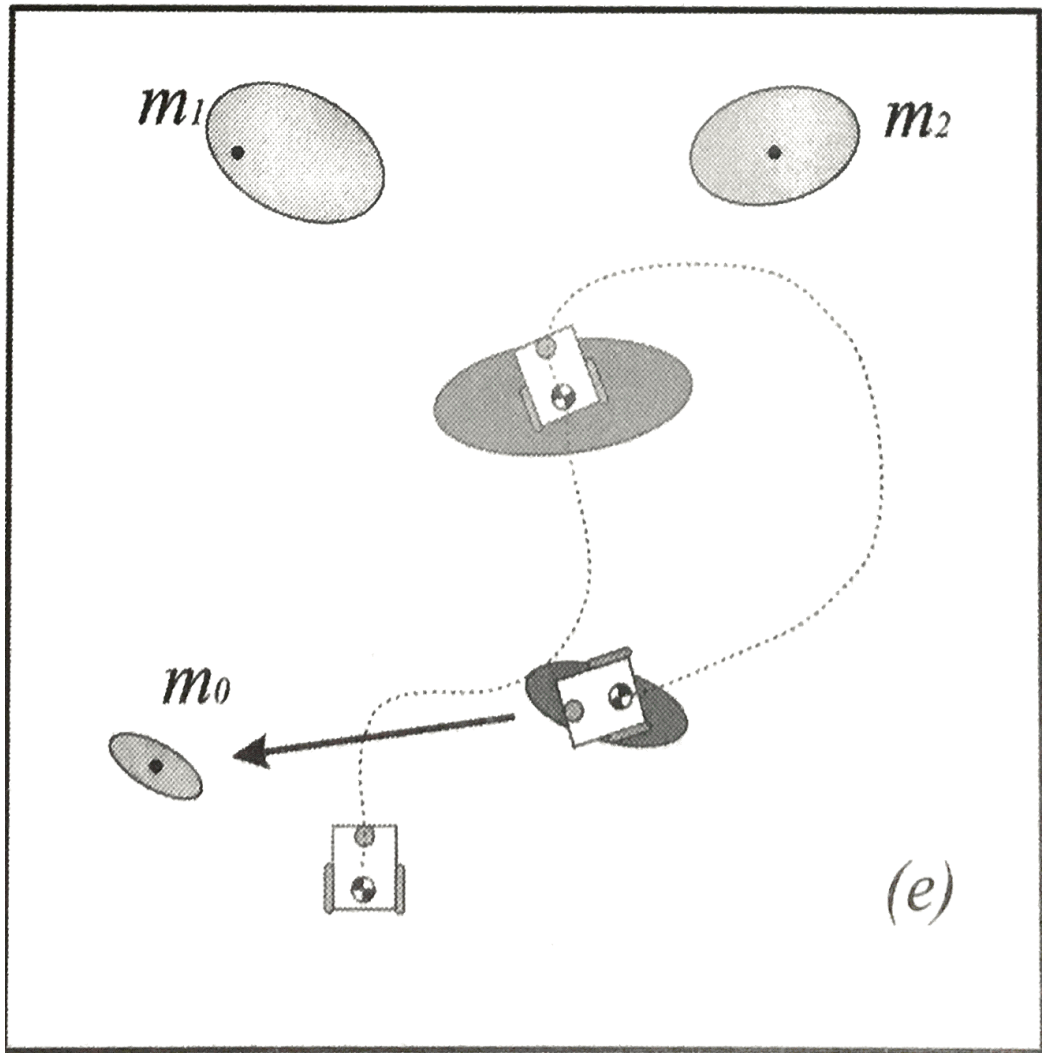
- Step (c)
  - The robot observes two landmarks m_1 and m_2, and maps them with an uncertainty that results from the **combination of the measurement error and robot pose uncertainty**.
  - Map is corrected with the robot position estimation.

SLAM 6



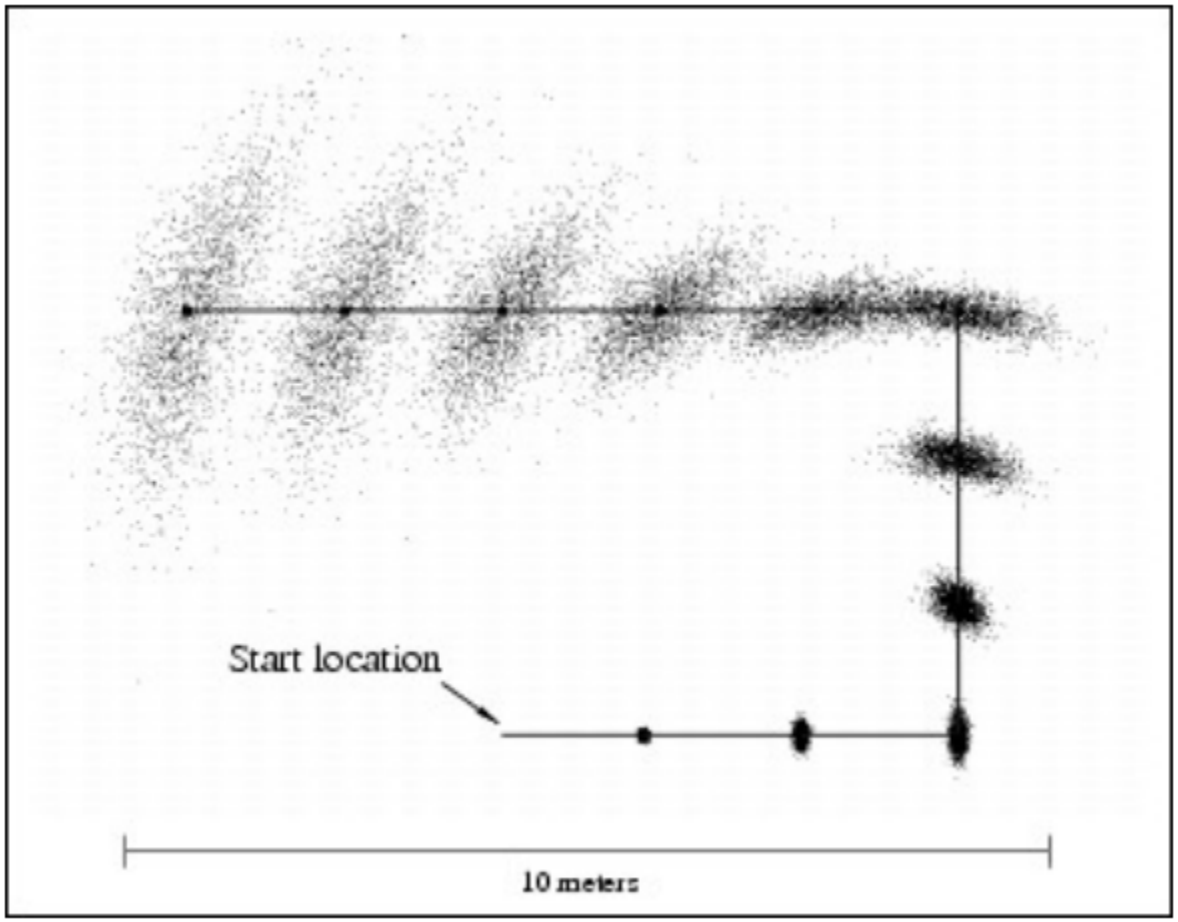
- Step (d)
  - The pose uncertainty further increases.
  - But the position can be updated based on the observation of two landmarks.
  - If the landmarks m_1 and m_2 are not precisely mapped, the robot's pose estimation can help building a better map.

SLAM 7



- Step (e)
  - The landmark m_0 is visited before, and can form a loop closure.
  - When a loop closure is detected, the robot pose uncertainty shrinks.
  - At the same time, the map can be updated.

Particle Filter



- **Random sampling + resampling**
- Particles are distributed randomly at the initial stage.
- Based on sensor readings, we can evaluate each particle: find the ones that have small deviations from sensor readings.
- **Resampling**: just keep the good particles in next cycle.
- The distribution of particles can be used to as uncertainty in SLAM.

Optional

- We don't have lidar or sonar, how to use SLAM?
  - We can use the camera
  - The scale-invariant, rotation-invariant features can be used: e.g. corner point detection, SIFT, etc.
  - Or even simpler: line detection + match making in frames

An Example (Optional)

- A SLAM project with leJOS EV3
  - https://lejosnews.wordpress.com/2017/07/15/slam/
    - Line tracker
    - Line follower
    - Map tracker
    - Status tracker
- Is it possible to use a camera and an ultrasonic sensor to build the map?